

WAKE: A Weakly supervised Business Process Anomaly Detection Framework via a Pre-trained Autoencoder

Wei Guan, Jian Cao, *Senior Member, IEEE*, Haiyan Zhao, Yang Gu, Shiyu Qian, *Member, IEEE*

Abstract—The ability to detect anomalies in business processes is crucial for achieving success in business operations. While unsupervised anomaly detection approaches have gained popularity in recent years due to their label-free nature, in some cases, a limited number of labelled anomalies can be provided and using them can improve the performance of anomaly detection. To address this issue, we propose a novel framework for anomaly detection that uses a pre-trained autoencoder to extract feature representations of traces. An anomaly score generator based on a multi-layer perceptron is utilized to evaluate the extracted features. The entire framework is trained using a joint loss that ensures the generated anomaly scores satisfy a specific distribution without compromising the autoencoder's ability to reconstruct normal traces. The feature encoder is fine-tuned to provide insights into the cause of anomalies. Additionally, we design a novel technique for calculating anomaly scores to mitigate the effects of varying numbers of potential attribute values. We conduct extensive experiments on both synthetic and real-life logs, and our results demonstrate that our proposed method, WAKE, outperforms state-of-the-art unsupervised deep business process anomaly detection methods by a significant margin. Additionally, it outperforms other weakly supervised anomaly detection methods as well.

Index Terms—Process mining, weakly supervised anomaly detection, deep learning, recurrent neural networks, autoencoder

1 INTRODUCTION

ENSURING smooth and organized business processes is crucial to maintaining a successful enterprise. This makes it an important task for both IT departments and business administrators. Despite efforts to maintain the correct executions of these processes, anomalies are inevitable and can arise from various causes such as system malfunctions and human errors. Detecting anomalies in event traces of business processes can aid practitioners and decision-makers in analyzing and managing businesses more effectively, and preventing severe consequences resulting from anomalies.

Many anomaly detection methods for business processes have been proposed, including conformance checking-based [1], [2], information theory-based [3], [4], probabilistic [5], [6], [7] and distance-based [8], [9]. They are either based on assumptions about the data distribution or require extensive prior knowledge of the process, which is not always available in real life.

In recent years, deep learning [10] has achieved remarkable feats in revealing complex relationships in high-dimensional data. Not surprisingly, it has become the mainstream approach for business process anomaly detection [11], [12], [13], [14], [15]. In these models, anomalies are detected in terms of reconstruction errors directly and there is no need to spend time annotating the data. However,

without incorporating prior knowledge about anomalies, the performance of these approaches and the reliability of their results can not be guaranteed.

Providing supervision information to train a model can not only improve its performance but also enhance its trustworthiness. Unfortunately, fully-supervised anomaly detection approaches are often impractical as they require labelled training data containing both normal and anomalous samples [16]. However, detecting anomalies using a limited number of labelled anomalies can be challenging. Firstly, the insufficient supervised information provided by the limited number of labelled anomalies poses a significant challenge to generalize the detection of both known anomalies, whose behaviors resemble those of labelled anomalies, and unknown anomalies, whose behaviors differ from those of labelled anomalies. Additionally, in the unlabelled samples, the large number of normal samples can potentially overwhelm the model's ability to distinguish between normal and anomalous behavior.

Thankfully, researchers have proposed several approaches to address these challenges. For instance, there has been research into anomaly detection with incomplete supervision, which involves labeling only a portion of the input data. The major methods [17], [18], [19] focus on learning feature representations while other approaches [20], [21], [22] directly learn anomaly scores within an end-to-end learning framework. Approximate labels can also be generated to detect anomalies by directly addressing the insufficiency of labels [23], [24], [25]. Moreover, the multiple-instance learning (MIL) framework is applied in [26], [27], [28] to detect anomalies with inexact labels.

Despite these advances, these weakly supervised anomaly detection methods for other applications are not

- W. Guan, J. Cao, Y. Gu and S. Qian are with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, 200240, P.R. China. E-mail: {guan-wei, cao-jian, gu_yang, qshiyu}@sjtu.edu.cn.
- H. Zhao is with the University of Shanghai for Science and Technology, Shanghai, China. Email: zhaohaiyan1992@foxmail.com

Manuscript received June 1, 2023; revised September 13, 2023. (Corresponding author: J. Cao.)

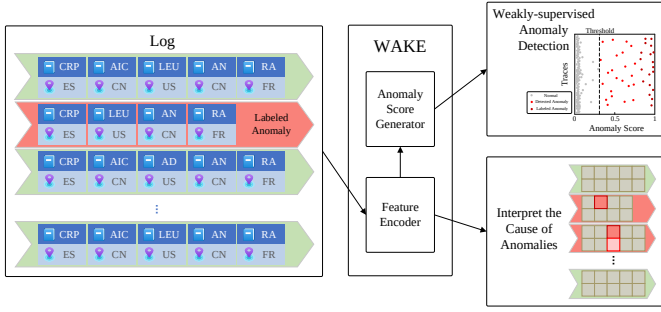


Fig. 1: Illustration of our proposed framework

directly applicable to business processes. Firstly, they do not have the ability to learn the temporal relationships between events in traces of business processes. Secondly, these methods require identical features as inputs, while traces have different lengths. Thirdly, none of these methods simultaneously consider incomplete and inexact supervision.

Recently, Elaziz et al. [29] introduced a novel weakly supervised anomaly detection method tailored for business processes. However, a notable limitation of their approach is the absence of explanations for the detected anomalies, which holds significant practical importance. Identifying which attribute of an event contributes to the detection of an anomaly using trace-level (inexact) labels can be challenging due to ambiguous label information.

To tackle the aforementioned challenges, we design a Weakly-supervised business process Anomaly detection framework via a pre-trained autoencoder, named WAKE, which is illustrated in Fig. 1. WAKE, which effectively exploits a few labelled anomalies in logs, is optimized through a three-stage training procedure to detect and interpret anomalies. WAKE leverages a pre-trained autoencoder capable of reconstructing normal traces as a feature encoder. The feature representation extracted from the feature encoder somehow reveals prior knowledge about the normal traces. Specifically, the feature representation of the trace is composed of three factors (i.e., latent representation, reconstruction error vector and maximum reconstruction error vector) and characterizes whether the trace deviates from the normal pattern. Latent representation is the low-dimensional vector representation of the trace, while reconstruction error vector and maximum reconstruction error vector are the reconstruction errors of the trace. Following feature extraction, the feature representations are fed into an anomaly score generator to detect anomalies. The feature encoder and the anomaly score generator are jointly optimized using both unlabelled traces and a limited number of labelled anomalies. The feature encoder can also be applied to interpret the cause of anomalies after fine-tuning to provide an insight into anomalies.

The main contributions of our work are as follows:

- We provide a formal definition for the problem of detecting and interpreting the cause of anomalies in business processes using weak supervision.
- We design the WAKE framework, which can detect and interpret the cause of anomalies by utilizing incomplete and inexact labelled anomalies.
- We propose a new method for extracting feature

representations of traces by utilizing a pre-trained autoencoder.

- To interpret the causes of anomalies, we propose a novel calculation method for anomaly scores that mitigates the influence of varying numbers of potential attribute values.

We conduct comprehensive experiments on synthetic and real-life datasets, which demonstrate that WAKE achieves superior performance. Additionally, we carry out ablation studies to evaluate the impact of each individual component in our approach.

The rest of this paper is organized as follows. Section 2 introduces the related work. Section 3 gives the definition of the problem. Then proposed framework WAKE is elaborated in section 4. Section 5 reports the experiment results on both synthetic and real-life logs. Finally, we conclude our work and discuss future work in section 6.

2 RELATED WORK

To align with the focus of our research, we provide a concise review of existing work in three areas: i) traditional business process anomaly detection approaches, ii) deep business process anomaly detection approaches, and iii) the utilization of weakly supervised learning in anomaly detection. To serve as a point of reference, we present an overview of these works in Table 1.

2.1 Traditional Business Process Anomaly Detection Approaches

Traditional business process anomaly detection approaches can be broadly divided into four categories.

The conformance checking-based approaches [1], [30] are the first category, which utilize conformance checking techniques to examine the synchronisation between the recorded log and the corresponding process model. The process model can either be given or discovered from the log using process mining techniques. In some studies [2], [31], the integration of conformance checking techniques and association rule learning has been shown to improve the accuracy of anomaly detection.

The second category is information theory-based approaches [3], [4], [32], [33], which applies statistical leverage to detect trace-level anomalies in business processes while accounting for varying trace lengths.

The third category is probabilistic approaches, which utilizes probabilistic graphical models such as likelihood graph [34], Bayesian networks [35], hidden Markov model (HMM) [5], [6] and variable order Markov models (VOMMs) [7] to create normal process behavior probabilistic models based on event logs for detecting anomalies.

The fourth category involves distance-based approaches that consider the distance between traces for anomaly detection. For example, Sureka [8] calculate the normalized longest common sub-sequence (NLCS) between traces and apply k nearest neighbor (KNN) to detect anomalies. Kang et al. [36] use the LOF algorithm based on k nearest neighbor imputation (KNNI) to detect abnormal termination. In other studies [37], outlier-aware clustering algorithms are used, while Hsu et al. [9] utilize the duration of events to

TABLE 1: A brief summary of related works.

Categories		Algorithms	Comments/Limitations
Traditional Business Process Anomaly Detection	Conformance Checking-based	[1], [2], [30], [31]	Requires a process model or a clean log. The quality of the process model significantly affects the performance of anomaly detection.
	Information theory-based	[3], [4], [32], [33]	Relies on data distribution assumptions that may not always be valid.
	Probabilistic	[5], [6], [7], [34] [35]	Requires a clean log for model learning. Unable to capture complex dependencies in the process.
	Distance-based	[8], [9], [36], [37]	Suffers from the curse of dimensionality.
Deep Business Process Anomaly Detection	Unsupervised	[38], [39], [40], [41] [11], [12], [42], [43] [13], [14], [15], [44]	Does not require any prior knowledge of the business process. Cannot make use of the limited number of labelled anomalies, which may potentially restrict its capacity to detect anomalies with precision.
	weakly supervised	[29], [45]	The deep model is trained under incomplete supervision.
Weakly supervised Anomaly Detection	Incomplete	[17], [18], [19], [46] [20], [21], [22], [47] [23], [24], [25], [48]	Only partial input data is labelled, making fully-supervised models inapplicable. WAKE can deal with this scenario as it only needs a few labelled anomalies.
	Inexact	[26], [27], [28], [49] [50], [51]	Only coarse-grained labels are available, but we require fine-grained labels. WAKE can detect attribute-level anomalies when only trace-level labels are available.
	Inaccurate	[52], [53], [54]	Labels are noisy, i.e., they may contain incorrect labels.
Masked Autoencoding Techniques		[55], [56], [57], [58] [59], [60], [61], [62]	Introduce a masking mechanism that selectively masks specific parts of the input during the encoding and decoding process.

compute the distance between traces for detecting temporal anomalies.

The traditional approaches are either based on data distribution assumptions or require prior knowledge, and none of them can take advantage of labelled anomalies.

2.2 Deep Business Process Anomaly Detection Approaches

Recently, deep business process anomaly detection have gained significant attention due to their ability to extract complex underlying relationships from unlabelled data. These techniques can be categorized into two main groups: *unsupervised* and *weakly supervised* approaches.

Unsupervised: Due to the user-friendly nature of unsupervised methods, they have garnered extensive research attention. Studies by Tavares et al. [38] and Junior et al., [39] encode traces as vectors using the word2vec [63] encoding method, taking into account semantic information about the process. After encoding, the random forest algorithm [38] or one-class SVM [39] can be applied to detect anomalous traces. Krajsic and Franczyk [40] use an autoencoder to reconstruct traces and detect anomalies based on reconstruction error. To overcome the overfitting problem, Nolle et al. [11], [41] utilize a denoising autoencoder that introduces additive Gaussian noise over the input vector. Nguyen et al. [12], [42] employ a variational autoencoder (VAE), which is an autoencoder with additional constraints on the hidden representation. Considering that traces are time series, the authors in [12], [13], [15], [43] embed LSTM or GRU in the autoencoder to improve the reconstruction capability of the model. GAE [14] and GAMA [44] convert traces into graphs and utilize graph neural networks (GNN) to obtain graph encoding. Anomalies are detected based on the reconstruction error of the graphs or traces.

These approaches may be biased by the presence of anomalies, as they treat normals and anomalous data equally.

weakly supervised: Krajsic et al. [45] propose a bidirectional LSTM variational autoencoder with self-attention. This approach requires a clean log for training purposes. In the study by Elaziz et al. [29], reinforcement learning is employed to enhance information extraction from a limited number of labelled anomalies.

Nevertheless, current approaches fall short in their ability to interpret the fundamental causes underlying anomalies.

2.3 Weakly supervised Learning for Anomaly Detection

Given the current limited research on applying weakly supervised learning to business process anomaly detection, we conduct a thorough review of weakly supervised learning approaches applied in anomaly detection within various other domains. These studies leverage labelled data to improve the anomaly detection performance, even though the supervision may be incomplete, inexact, or inaccurate.

Incomplete supervision refers to the situation where only part of the input data is labelled. Some studies [17], [18], [19], [46], [47], [48] have utilized deep neural networks to extract anomaly-oriented representations from input data. While feature representation learning can enhance downstream detection, it only provides indirect learning for anomaly scores. End-to-end anomaly score learning frameworks such as DevNet [20], PReNet [21], and FEAWAD [22] can directly acquire anomaly scores from trained models. In addition to using existing labels, active learning strategies are applied in [23], [24], [25] to gather more supervision.

Anomaly detection with inexact supervision occurs when labels are not as precise as required, i.e., only coarse-grained labels are available due to the high cost of labeling.

In [26], [27], [28], [49], authors detect anomalies with inexact supervision via the multiple-instance learning (MIL) framework, where video segments are considered as instances, while normal and abnormal videos are classified as positive and negative bags, respectively. A self-reasoning framework proposed in [50] trains neural networks by generating pseudo labels through binary clustering of video features. WETAS [51] uses dynamic time wrapping (DTW) alignment to enhance detection performance.

Anomaly detection with inaccurate supervision occurs when labels are not entirely accurate and may contain errors. Zhang et al. and Zhao et al. [52], [53] propose to learn a joint model from multiple sets of noisy labels using ensemble learning. Gao et al. [54] propose a denoising network based on GAN for anomaly detection in dynamic graphs that employs self-training to progressively refine inaccurate node labels.

Our proposed approach is superior to these methods, as it can detect anomalies with both incomplete and inexact supervision in business processes.

2.4 Masked Autoencoding Techniques

Masked autoencoders (MAE) have recently gained significant traction as a scalable self-supervised learning method. The seminal paper by He et al. [55] introduces masked autoencoders for self-supervised learning of image representations. This fundamental contribution has paved the way for their exploration in various domains. For example, in [56], [57], MAEs are effectively employed for learning graph representations. Notably, Huang et al. [58] extend the application of MAEs to the realm of audio data.

Notably, MAEs have found utility in anomaly detection. The authors in [59], [60] have harnessed MAEs to identify image anomalies. Furthermore, the capabilities of MAEs extend to video anomaly detection [61], and even to the detection of anomalies in road networks [62].

3 PROBLEM FORMULATION

To set the foundation for the following sections, we will provide definitions for key terms including *attribute*, *event*, *trace*, and *log*.

Definition 3.1 (Event, Trace, Log). Let $\mathcal{A} = \{a_1, a_2, \dots, a_A\}$ denote a set of attributes, where $A = |\mathcal{A}|$ represents the total number of attributes. For each attribute $a \in \mathcal{A}$, $\mathcal{V}_a$ is the set of possible values. An *event* is a tuple $e = [V_{a_1}, V_{a_2}, \dots, V_{a_A}]$, where $V_a \in \mathcal{V}_a$ is a potential value for attribute a . A *trace* t is a sequence of events, and an *event log* $\mathcal{L}$ is a set of traces. We denote the number of events in a trace t by $|t|$, and the number of traces in an event log $\mathcal{L}$ by $|\mathcal{L}|$.

Please note that *activity* is a unique attribute in the attribute set $\mathcal{A}$ that reflects the control flow. Let $t_{e,a}$ denote the value of attribute a of event e in trace t .

Example 1. Given an attribute set $\mathcal{A} = \{\text{Activity}, \text{Country}\}$. $t = \langle [\text{CRP}, \text{ES}], [\text{AIC}, \text{CN}], [\text{LEU}, \text{US}], [\text{AN}, \text{CN}], [\text{RA}, \text{FR}] \rangle$ represents a trace containing five simple events taking place in chronological order. The value of the first attribute (i.e., *Activity*) in the second event (i.e., $[\text{AIC}, \text{CN}]$) in trace t can be obtained with $t_{2,1} = \text{AIC}$.

Next, we define the weakly supervised (i.e., with incomplete supervision) business process anomaly detection as follows:

Definition 3.2 (weakly supervised Business Process Anomaly Detection). Given a log $\mathcal{L} = \langle t_1, t_2, \dots, t_u, t_{u+1}, t_{u+2}, \dots, t_{u+g} \rangle$ containing $u + g$ traces, where $\mathcal{U} = \langle t_1, t_2, \dots, t_u \rangle$ is a set of unlabelled traces and $\mathcal{G} = \langle t_{u+1}, t_{u+2}, \dots, t_{u+g} \rangle$ with $g \ll u$ is a very small set of labelled anomalies that provide some prior knowledge of anomalies, the goal is to detect some anomalous traces that differ significantly from the majority of traces in $\mathcal{U}$.

However, it is not sufficient to only detect these anomalies, we also need to interpret the cause of anomalies (i.e., with inexact supervision), which are defined as follows:

Definition 3.3 (Interpretation of the Cause of Anomalies). Given a set of labelled anomalies $\mathcal{G}$ and a set of detected anomalous traces $\mathcal{D}$, which is a subset of unlabelled trace set, the goal is to determine which attribute a of which event e in trace $t \in \mathcal{G} \cup \mathcal{D}$ is anomalous.

4 METHODOLOGY

In this section, we introduce our proposed anomaly detection framework, WAKE, which employs a novel feature encoding strategy to overcome the limitations of inadequately labelled anomalies. Fig. 2 shows the architecture of WAKE, which comprises an autoencoder-like feature encoder and an anomaly score generator that collaborate with each other within the model. The feature encoder converts the trace into a more meaningful feature representation, i.e., three factors that characterize whether the trace deviates from the normal pattern, while the anomaly score generator generates an anomaly score based on the feature representation from the feature encoder. The anomaly score is a scalar between 0 and 1 indicating the probability that the trace is an anomaly. WAKE is optimized through a three-stage training procedure to detect and interpret the cause of anomalies in the log. The following subsections provide details about the proposed framework.

4.1 Feature Encoder

The objective of the feature encoder is to transform the input trace into a meaningful feature representation that improves anomaly detection. To accomplish this, we utilize an autoencoder (AE) with recurrent layers to generate the feature representation. Particularly, three factors, namely the latent representation, reconstruction error vector, and maximum reconstruction error vector, characterize whether the trace deviates from the normal pattern. In addition, we employ teacher forcing methods to improve the convergence speed and reconstruction capability of the feature encoder.

In order to make the model work properly, we pad the traces that are shorter than E with padding events to make their length equal to E , where $E = \max_{t \in \mathcal{L}} |t|$ is the maximum length of traces in log $\mathcal{L}$. As different attributes have distinct anomaly patterns, we assign different embedding layers, bidirectional gated recurrent units (BGRU) encoders, attention layers, gated recurrent units (GRU) [64] decoders, linear layers, and softmax, to different attributes.

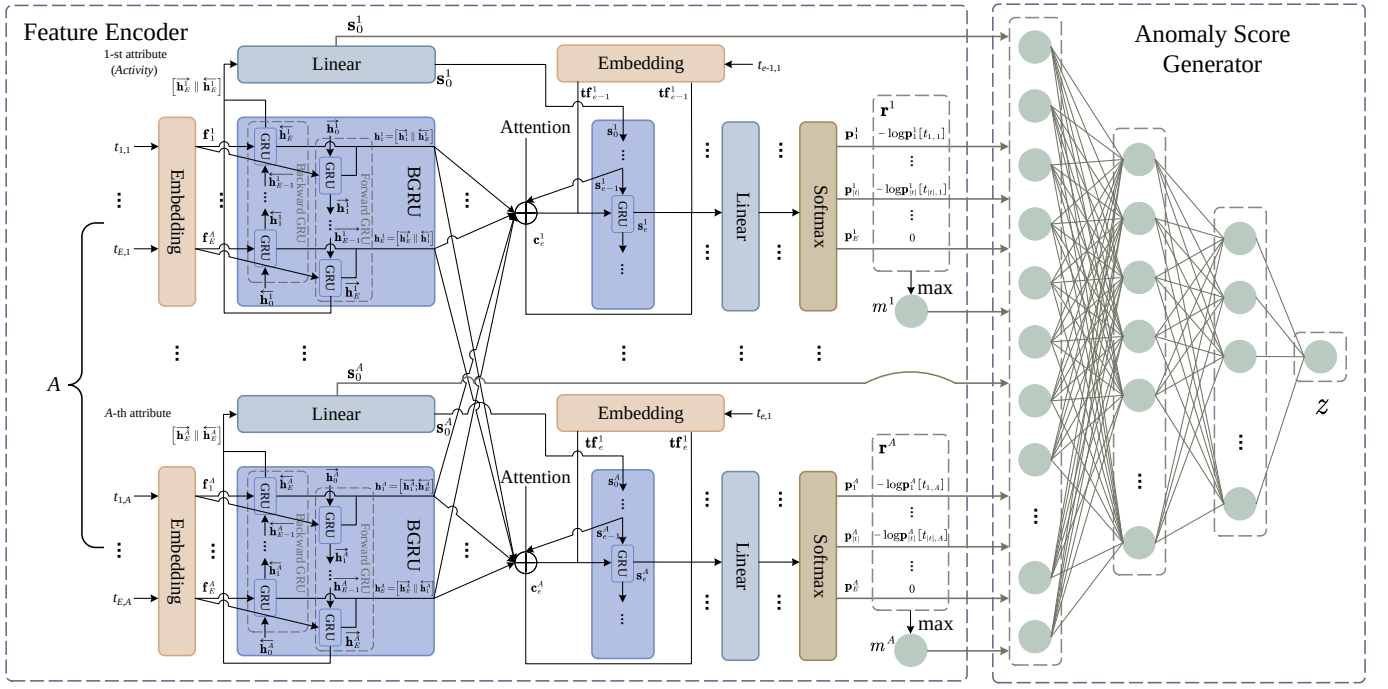


Fig. 2: The architecture of WAKE where $E = \max_{t \in \mathcal{L}} |t|$ is the maximum length of traces in $\log \mathcal{L}$ and A is the total number of attributes.

Due to the mathematical nature of neural networks, embedding layers are used to convert each attribute value $t_{e,a}$ into vector representation $\mathbf{f}_e^a$. GRUs typically learn information from previous states. However, in cases where the current attribute value is influenced by both the attribute value preceding it and the one following it, the conventional learning behavior of GRUs proves to be insufficient for accurately reconstructing the attribute value. Therefore, the embedding vectors $\mathbf{f}_e^a$ are fed into the BGRU, which is an extension of GRU. The BGRU consists of two GRUs: a forward GRU that accepts forward inputs and a backward GRU that accepts backward inputs. For each attribute a of event e , the forward GRU generates forward hidden states $\overrightarrow{\mathbf{h}}_{e,e}^a$ and the backward GRU generates backward hidden states $\overleftarrow{\mathbf{h}}_{E+1-e}^a$. These hidden states are concatenated to produce the BGRU output $\mathbf{h}_e^a = [\overrightarrow{\mathbf{h}}_{e,e}^a \| \overleftarrow{\mathbf{h}}_{E+1-e}^a]$. To generate the latent representation $\mathbf{s}_0^a$ based on the autoencoder for attribute a , the final hidden states of the forward GRU $\overrightarrow{\mathbf{h}}_{e,e}^a$ and the backward GRU $\overleftarrow{\mathbf{h}}_{E+1-e}^a$ are concatenated and fed into linear layers. The activation function used in the linear layer is the Leaky ReLU.

$$\mathbf{s}_0^a = \text{LeakyReLU}(\mathbf{W}_s^a [\overrightarrow{\mathbf{h}}_{e,e}^a \| \overleftarrow{\mathbf{h}}_{E+1-e}^a]) \quad (1)$$

where $\mathbf{W}_s^a$ is a learnable matrix and $\text{LeakyReLU}(x) = \max(0.1x, x)$ is a non-linear activation function. The first factor of our feature representation is the concatenation of $\mathbf{s}_0^a$ for all attributes $a \in \mathcal{A}$:

$$\mathbf{s} = \parallel_{a \in \mathcal{A}} \mathbf{s}_0^a \quad (2)$$

where $\parallel$ indicates concatenation.

We will now explain the process of reconstructing the value of attribute a of event e . To capture the inherent relationships among attributes, all the outputs of the BGRU

encoders, $\mathbf{H} = \{\mathbf{h}_1^1, \mathbf{h}_2^1, \dots, \mathbf{h}_{E-1}^A, \mathbf{h}_E^A\}$, must be fed into the attention layer:

$$eg_{e,i}^a = (\mathbf{V}_\alpha^a)^T \tanh(\mathbf{W}_\alpha^a [\mathbf{s}_{e-1}^a \| \mathbf{h}_i]) \quad (3)$$

$$\alpha_{e,i}^a = \frac{\exp(eg_{e,i}^a)}{\sum_m \exp(eg_{e,m}^a)} \quad (4)$$

$$\mathbf{c}_e^a = \sum_i \alpha_{e,i}^a \mathbf{h}_i \quad (5)$$

where $\mathbf{h}_i \in \mathbf{H}$, $\cdot^T$ represents transposition, $\mathbf{W}_\alpha^a$ is a learnable matrix, $\mathbf{V}_\alpha^a$ is a learnable vector and $\mathbf{s}_{e-1}^a$ is the hidden state of the GRU decoder for attribute a at the previous event $e-1$.

To improve the feature encoder's reconstruction capability, we utilize the teacher forcing method, which is a widely used technique in natural language processing. For the first attribute (Activity), we use the previous ground truth attribute value (i.e., activity name) $t_{e-1,1}$ to guide the reconstruction of the activity name for the current event e .

$$\mathbf{s}_e^1 = \text{GRU}^1(\mathbf{s}_{e-1}^1, [\mathbf{c}_e^1 \| \mathbf{tf}_{e-1}^1]) \quad (6)$$

where $\mathbf{s}_{e-1}^1$ is the hidden state of the first GRU decoder for the first attribute at previous event $e-1$, $\mathbf{tf}_{e-1}^1$ is the embedding vector of $t_{e-1,1}$ and $\mathbf{c}_e^1$ is the output of the attention layer for the first attribute at event e . The initial hidden state of the first GRU decoder for the first attribute is $\mathbf{s}_0^1$. The probability distribution $\mathbf{p}_e^1$, which is the probability distribution over all possible values of the first attribute (Activity) of event e , can be calculated by

$$\mathbf{p}_e^1 = \text{Softmax}(\mathbf{W}_p^1 [\mathbf{s}_e^1 \| \mathbf{c}_e^1 \| \mathbf{tf}_{e-1}^1]) \quad (7)$$

which represents the linear layer and the softmax layer.

We assume that the value of the other attribute a is primarily affected by the current activity name. Thus, we

use the ground truth activity name $t_{e,1}$ of the current event e to predict the probability distribution over all possible values of attribute a of event e . The probability distribution $\mathbf{p}_e^a$ can be computed by

$$\mathbf{s}_e^a = \text{GRU}^a(\mathbf{s}_{e-1}^a, [\mathbf{c}_e^a \parallel \mathbf{tf}_e^1]) \quad (8)$$

$$\mathbf{p}_e^a = \text{Softmax}(\mathbf{W}_p^a[\mathbf{s}_e^a \parallel \mathbf{c}_e^a \parallel \mathbf{tf}_e^1]) \quad (9)$$

where $\mathbf{tf}_e^1$ is the embedding vector of $t_{e,1}$.

Specifically, $\mathbf{p}_e^a$ is a vector of dimension $|\mathcal{V}_a|$. Let $\mathbf{p}^a = [\mathbf{p}_1^a[t_{1,a}], \dots, \mathbf{p}_{|t|}^a[t_{|t|,a}], 1, \dots, 1]$, where $|t|$ is the original length of trace t and $\mathbf{p}_e^a[t_{e,a}]$ denotes the probability that the value of attribute a of event e in trace t is $t_{e,a}$. The probability vector can be represented as

$$\mathbf{p} = \parallel_{a \in \mathcal{A}} \mathbf{p}^a \quad (10)$$

Take the negative logarithm of each element in $\mathbf{p}^a$, we can obtain $\mathbf{r}^a = [-\log \mathbf{p}_1^a[t_{1,a}], \dots, -\log \mathbf{p}_{|t|}^a[t_{|t|,a}], 0, \dots, 0]$. Then, the second factor of the feature representation, i.e., the reconstruction error vector can be represented as

$$\mathbf{r} = \parallel_{a \in \mathcal{A}} \mathbf{r}^a \quad (11)$$

Finally, the third factor of the feature representation, i.e., the maximum reconstruction error vector is $\mathbf{m} = [\max(\mathbf{r}^1), \dots, \max(\mathbf{r}^A)]$ where $\max(\mathbf{r}^a)$ represents the maximum value in the vector $\mathbf{r}^a$.

In summary, the input trace is characterized by the three factors (i.e., latent representation $\mathbf{s}$, reconstruction error vector $\mathbf{r}$ and maximum reconstruction error vector $\mathbf{m}$) and anomalies in any one of these factors will be detected by WAKE. Each of these three factors has its own specific meaning. $\mathbf{s}$ is the low-dimensional vector representation of the trace, while $\mathbf{r}$ and $\mathbf{m}$ are the reconstruction errors of the trace, which have been shown to reflect anomalies in previous studies [11], [12], [40].

4.2 Anomaly Score Generator

The anomaly score generator produces a scalar, i.e., an anomaly score, between 0 and 1 based on the feature representation from the feature encoder. The anomaly score indicates the probability that a trace is anomalous.

The concatenation of three factors is processed by a multi-layer perceptron (MLP) to map the input into an anomaly score. Specifically, for each layer of the MLP, we have

$$\mathbf{z}_k = \text{LeakyRELU}(\mathbf{W}_z^k \mathbf{z}_{k-1} + b_k) \quad (12)$$

where $\mathbf{z}_k$ and $\mathbf{z}_{k-1}$ are the activations at the k -th layer and $k-1$ -th layer, respectively. $\mathbf{W}_z^k$ and b_k are the weight and bias for the k -th layer. For the first layer of MLP, $\mathbf{z}_0 = [\mathbf{s} \parallel \mathbf{r} \parallel \mathbf{m}]$. For the last layer, we use the sigmoid activation function to map $\mathbf{z}_k$ to a scalar between 0 and 1, producing the anomaly score z .

4.3 Training Procedure

The training procedure of WAKE consists of three stages: i) Pre-training: The feature encoder is trained to reconstruct normal traces. ii) End-to-end optimization: The feature encoder and the anomaly score generator are jointly trained based on the pre-trained feature encoder. iii) Fine-tuning:

Algorithm 1 Training Procedure

Stage 1: Pre-training

Input: Unlabelled trace set $\mathcal{U}$

Output: Feature encoder $\psi(\cdot; \Theta_{fe})$

```

1: Randomly initialize  $\Theta_{fe}$ 
2: for  $i = 1$  to  $n_{epochs_1}$  do
3:   for  $j = 1$  to  $n_{batches_1}$  do
4:      $\mathcal{B} \leftarrow$  Randomly sample  $batch\_size$  traces from  $\mathcal{U}$ 
5:      $loss \leftarrow$  Calculate  $L_{pre}(\mathcal{B})$  from Equation (13)
6:     Perform a gradient descent step w.r.t. the parameters
       in  $\Theta_{fe}$ 
7:   end for
8: end for
9: return  $\psi(\cdot; \Theta_{fe})$ 

```

Stage 2: End-to-end optimization

Input: Unlabelled trace set $\mathcal{U}$, labelled anomaly set $\mathcal{G}$

Output: Feature encoder $\psi(\cdot; \Theta_{fe})$, anomaly score generator $\varphi(\cdot; \Theta_{asg})$

```

10: Randomly initialize  $\Theta_{asg}$  and load  $\Theta_{fe}$  from the pre-
    trained feature encoder  $\psi(\cdot; \Theta_{fe})$  returned in stage 1
11: for  $i = 1$  to  $n_{epochs_2}$  do
12:   for  $j = 1$  to  $n_{batches_2}$  do
13:      $\mathcal{B} \leftarrow$  Randomly sample  $batch\_size$  traces from  $\mathcal{G}$  and
        $\mathcal{U}$  with probability  $\beta$  and  $1 - \beta$ , respectively
14:      $loss \leftarrow$  Calculate  $L_{ete}(\mathcal{B})$  from Equation (16)
15:     Perform a gradient descent step w.r.t. the parameters
       in  $\Theta_{fe}$  and  $\Theta_{asg}$ 
16:   end for
17: end for
18: return  $\psi(\cdot; \Theta_{fe})$ ,  $\varphi(\cdot; \Theta_{asg})$ 

```

Stage 3: Fine-tuning

Input: Entire log $\mathcal{L} = \mathcal{U} \cup \mathcal{G}$

Output: Another feature encoder $\psi'(\cdot; \Theta'_{fe})$ for interpreting the cause of anomalies

```

19:  $\Theta'_{fe} \leftarrow$  Load  $\Theta_{fe}$  from the feature encoder  $\psi(\cdot; \Theta_{fe})$ 
    returned in stage 2
20: for  $i = 1$  to  $n_{epochs_3}$  do
21:   for  $j = 1$  to  $n_{batches_3}$  do
22:      $\mathcal{B} \leftarrow$  Randomly sample  $batch\_size$  traces from  $\mathcal{L}$ 
23:      $loss \leftarrow$  Calculate  $L_{ft}(\mathcal{B})$  from Equation (17)
24:     Perform a gradient descent step w.r.t. the parameters
       in  $\Theta'_{fe}$ 
25:   end for
26: end for
27: return  $\psi'(\cdot; \Theta'_{fe})$ 

```

The end-to-end optimized feature encoder is fine tuned to enable it to interpret the cause of anomalies. The architecture parameters are trained using mini-batch stochastic gradient descent (SGD) and optimized with Adam [65]. We use $\psi(\cdot; \Theta_{fe})$ to denote a feature encoder and $\varphi(\cdot; \Theta_{asg})$ to denote an anomaly score generator, where Θ_{fe} and Θ_{asg} represent the model parameters for the feature encoder and the anomaly score generator, respectively. The training procedure is detailed in Algorithm 1.

It is worth noting that our anomaly detection framework is weakly supervised, meaning that only a limited number of labelled anomalies are available. The set of unlabelled traces may contain both normal and anomalous traces. However, as anomalies are rare in real-world and the majority of unlabelled traces are normal, a straightforward strategy is employed: considering all unlabelled traces as normal. This approach has been demonstrated to be effective in previous studies [20], [22]. Our experiments confirm that this simple strategy yields satisfactory performance.

4.3.1 Pre-training

We first employ a pre-training stage to train the feature encoder to be able to reconstruct normal traces, providing a basic reference for the subsequent end-to-end optimization. The pre-training stage is depicted in lines 1-9 of Algorithm 1. The feature encoder is pre-trained with all unlabelled traces by minimizing the reconstruction errors (i.e., cross-entropy loss). Our objective is to reconstruct the value of each attribute a of each event e in each trace $t \in \mathcal{B}$, where $\mathcal{B}$ is a batch of traces:

$$L_{pre}(\mathcal{B}) = \sum_{t \in \mathcal{B}} \sum_{r_i \in \mathbf{r}_t} r_i \quad (13)$$

where $\mathbf{r}_t$ represents the second factor of the feature representation of trace t (i.e., the reconstruction error vector).

4.3.2 End-to-end optimization

In stage 2, to ensure that WAKE was able to effectively detect anomalies, a mini-batch is constructed by random sampling, i.e., we sample *batch_size* traces from labelled anomaly set $\mathcal{G}$ and unlabelled trace set $\mathcal{U}$ with user-chosen probability β and $1 - \beta$, respectively, to form a mini-batch. In other words, the anomalies are oversampled to address the sample imbalance. The end-to-end optimization stage is depicted in lines 10-18 of Algorithm 1.

Our objective is to make the anomaly scores generated by the anomaly score generator satisfy a specific distribution (i.e., the anomaly scores of normal traces converge to 0 and the anomaly scores of anomaly traces converge to 1.) without destroying the ability of the feature encoder to reconstruct normal traces. Therefore, the first part of the loss function is:

$$L_{asg}(\mathcal{B}) = \sum_{t \in \mathcal{B}} [(1 - y_t)z_t + y_t(\log z_t)^2] \quad (14)$$

where $y_t \in \{0, 1\}$ indicates whether the current trace t is anomalous ($y_t = 1$) and z_t represents the anomaly score of trace t . In L_{asg} , we employ different loss functions depending on the trace's label. For an unlabelled trace, the loss increases linearly as its anomaly score increases. However, for a labelled anomalous trace, we utilize a logarithmic function and square to achieve an exponential increase in loss as its abnormal score decreases. The square operation is applied to introduce a varying degree of monotonicity to vary the monotonicity of the logarithmic function between 0 and 1. The primary objective of this equation is to assign a significantly higher loss to traces that have been confidently labelled as anomalous. By incorporating the logarithmic function, we effectively amplify the loss for these traces, magnifying their impact on the overall loss computation. This approach underscores the importance of accurately identifying and penalizing labelled anomalies, as they are deemed to be more critical and necessitate greater attention during the training procedure.

The second part of the loss function is to minimize the reconstruction errors for normal traces, and minimize the minimum probabilities of attribute values in anomalous traces, which is based on the idea that at least one attribute value in the anomalous trace is anomalous.

$$L_{re}(\mathcal{B}) = \sum_{t \in \mathcal{B}} [(1 - y_t) \sum_{r_i \in \mathbf{r}_t} r_i + y_t \min_{p_i \in \mathbf{p}_t} \{p_i\}] \quad (15)$$

where $\mathbf{r}_t$ represents the second factor of the feature representation of trace t . $\mathbf{p}_t$ represents the probability vector of trace t .

Based on Equation (14) and Equation (15), our joint loss function can be formally calculated as

$$L_{ete}(\mathcal{B}) = L_{asg}(\mathcal{B}) + \lambda L_{re}(\mathcal{B}) \quad (16)$$

where λ is a hyper-parameter to balance the contributions of two parts to the joint loss function.

4.3.3 Fine-tuning

Finally, we fine-tune the feature encoder obtained in stage 2 to obtain a new feature encoder so that it can interpret the cause of anomalies. The fine-tuning stage is depicted in lines 19-27 of Algorithm 1. We use the entire log $\mathcal{L} = \mathcal{U} \cup \mathcal{G}$, which contains both labelled anomalous traces and unlabelled traces without oversampling anomalies. The loss function in stage 3 is the same as that in Equation (15).

$$L_{ft}(\mathcal{B}) = L_{re}(\mathcal{B}) \quad (17)$$

4.4 Interpretation of the Cause of Anomalies

The output of stage 3 (i.e., feature encoder) can be applied to interpret the cause of anomalies. With the input trace t , we consider the probability distribution $\mathbf{p}_e^a$ over all possible values of attribute a of event e generated by the feature encoder.

Intuitively, compared to a normal attribute value, the probability of an anomalous one is lower. However, since the number of possible values for each attribute varies (i.e., $|\mathcal{V}_{a_1}| \neq |\mathcal{V}_{a_2}|$ where $a_1, a_2 \in \mathcal{A}$), it would be unfair to rely solely on a linear transformation of the probability of attribute values when calculating the anomaly score for each attribute of each event. To mitigate this phenomenon, we design a novel anomaly score that is comparable across attributes. We consider the logarithm of the probability of the value of attribute a in event e to be a random variable, then the difference between the expectation of this random variable (i.e., $\sum_{p_i \in \mathbf{p}_e^a} p_i \log p_i$) and the logarithm of the probability of the actual attribute value (i.e., $\log \mathbf{p}_e^a[t_{e,a}]$) is defined as the anomaly score, which is similar to information gain in decision tree [66]. The introduction of the logarithm increases the anomaly score for low-probability attribute values, while this expectation compensates for the effect of the difference in the number of possible attribute values. The anomaly score z_e^a for attribute a of event e can be calculated by:

$$z_e^a = \text{Sigmoid}(\sum_{p_i \in \mathbf{p}_e^a} p_i \log p_i - \log \mathbf{p}_e^a[t_{e,a}]) \quad (18)$$

where p_i is the i -th element of vector $\mathbf{p}_e^a$. If the actual attribute value has a low probability compared to the probability distribution, then the anomaly score will be high.

4.5 Complexity Analysis

Regarding the feature encoder, the computational complexity of the embedding layer is $O(EAxh)$, where h represents the hidden layer size and $x = \max\{|\mathcal{V}_a|\}_{a \in \mathcal{A}}$ is the maximum of the cardinalities of sets $\mathcal{V}_a$ for all a in $\mathcal{A}$. The computational complexity of the BGRU layer is $O(EAh^2)$.

The linear layer used to obtain s_0 has a computational complexity of $O(Ah^2)$. The attention mechanism introduces a complexity of $O(E^2A^2h^2)$. The computational complexity of the GRU layer is $O(EAh^2)$, and the linear layer before the Softmax has a complexity of $O(EAxh)$. Note that x is usually smaller than EAh , the overall time complexity is $O(E^2A^2h^2)$.

Moving on to the anomaly score generator, the input dimension is the sum of the dimensions of vectors s , p and m . The dimensions of vectors s , p and m are Ah , AE , and A respectively. Note that, the dimension of the hidden layer size is always smaller than the input dimension, the computational complexity of the anomaly score generator is $O(A(E+h)^2) = O(AE^2)$ or $O(Ah^2)$ based on the relative order of magnitude of E and h .

Overall, when considering an event log comprising T traces, whether it pertains to the optimization process of the feature encoder in stages 1 and 3, or the joint training of both the feature encoder and the anomaly score generator in stage 2, the time complexity of each epoch is $O(TE^2A^2h^2)$.

5 EXPERIMENTS

In this section, we empirically evaluate the effectiveness of WAKE on both synthetic and real-life datasets. WAKE is implemented in Python, and the source code is accessible at <https://github.com/guanwei49/WAKE>.

5.1 Compared Methods

To verify the superiority of the proposed method, we compare WAKE with ten state-of-the-art anomaly detection methods: DAE [11], VAE [12], LAE [12], BINet [13], GAE [14], DeepSAD [17], REPEN [18], DevNet [20], PReNet [21] and FEAWAD [22].

Among these methods, GAE, DAE, VAE, LAE and BINet are deep unsupervised anomaly detection methods specifically for business processes. They apply different types of autoencoders to reconstruct traces and detect anomalies based on reconstruction errors. DeepSAD and REPEN are weakly supervised anomaly detection methods by extracting an anomaly-oriented representation. DevNet, PReNet and FEAWAD are also weakly supervised anomaly detection methods by directly learning anomaly scores. DAE, VAE, LAE and BINet are adapted to perform the interpretation of the cause of anomalies. However, the other methods are not proposed for business processes and they cannot interpret the cause of the anomalies.

5.2 Parameter Settings and Metrics

WAKE is implemented based on PyTorch. In our experiment, we apply two-layer BGRUs for encoding and two-layer GRUs for decoding in feature encoder. Dropout is applied after each layer of both BGRUs and GRUs in the feature encoder and after each layer of the anomaly score generator to counteract overfitting. We initialize weights using the initialization introduced in [67], and train the proposed model for 14 epochs in stage 1, 10 epochs in stage 2 and 4 epochs in stage 3 (i.e., $n_epochs_1 = 14$, $n_epochs_2 = 10$, $n_epochs_3 = 4$) with Adam optimizer [65]. In the absence of special statements, the hidden layer size of the BGRUs

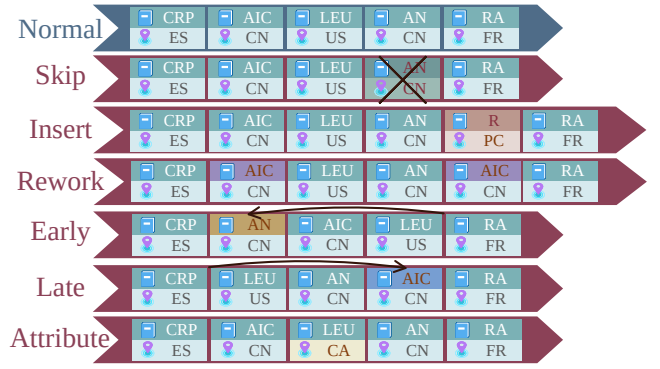


Fig. 3: Different anomaly types applied to a normal trace.

and GRUs is 64, the learning rate is 0.0002, the batch size is 64, β used to handle the sample imbalance in stage 2 is 0.3 and λ in the joint loss is 10. To ensure a fair comparison, the parameters for all the compared methods are set according to the values given in the original article. For the weakly supervised anomaly detection methods, we take ten percent of all anomalous traces as labelled anomalies and the other traces are unlabelled.

We employ the commonly used metric F -score which is the harmonic mean of $Precision$ and $Recall$ to evaluate the effectiveness of these methods. $Precision$ and $Recall$ are defined as $Precision = \frac{TP}{TP+FP}$, $Recall = \frac{TP}{TP+FN}$, and $F-score = \frac{2 * Precision * Recall}{Precision + Recall}$, respectively, where TP , FN and FP denote true positives, false negatives and false positives. The F -score for each method is calculated based on the optimal threshold value, which is determined by using a grid search method. The method has excellent anomaly detection performance when F -score is close to 1. We conduct each experiment five times and report the mean F -score.

5.3 Artificial Anomalies

Similar to previous studies [3], [11], [13], [30], [34], we create artificial anomalies in logs to evaluate the effectiveness of WAKE. We manually inject six types of anomalies into the logs, which are commonly observed in real-life business processes [13]. Fig. 3 illustrates the different anomalous traces generated by applying the six types of anomalies to a normal trace. The definitions of these anomaly types are as follows:

- *Skip*: A sequence of events has been skipped.
- *Insert*: A sequence of random events has been inserted.
- *Rework*: A sequence of events has been executed a second time.
- *Early*: A sequence of events has been executed too early, and hence is skipped later.
- *Late*: A sequence of events has been executed too late, and hence is skipped earlier.
- *Attribute*: Some attribute value has been replaced by an incorrect value in some events.

5.4 Dataset

We evaluate our method using both synthetic and real-life logs. To generate synthetic logs, we utilize simulation

TABLE 2: Statistics of the logs used in the experiments.

Log	#Activities	#Traces	#Events	Max trace length	Min trace length	#Attributes	#Attribute values
Gigantic	76-78	5000	28243-31989	11	3	1-4	70-363
Huge	54	5000	36377-42999	11	5	1-4	69-340
Large	42	5000	51099-56850	12	10	1-4	68-292
Medium	32	5000	28416-31372	8	3	1-4	66-276
P2p	13	5000	37941-42634	11	7	1-4	39-146
Paper	14	5000	49839-54390	12	9	1-4	36-128
Small	20	5000	42845-46060	10	7	1-4	39-144
Wide	23-34	5000	29128-31228	7	5-6	1-4	53-264
Approval	36	13087	262200	175	3	0	0
Loan	26	31509	1202267	180	10	1	149
Declaration	17	10500	56437	24	1	2	9
Prepaid	29	2099	18246	21	1	2	10
Request	19	6886	36796	20	1	2	10
Billing	18	100000	451359	217	1	0	0
Receipt	27	1434	8577	25	1	2	58
Sepsis	16	1050	15214	185	3	1	26

TABLE 3: F -scores over synthetic logs where 'AD' represents weakly supervised business process anomaly detection and 'IA' represents interpretation of the cause of anomalies. The best results are indicated using bold typeface.

Methods \ Logs	Paper		P2P		Small		Medium		Large		Huge		Gigantic		Wide	
	AD	IA	AD	IA	AD	IA	AD	IA	AD	IA	AD	IA	AD	IA	AD	IA
WAKE	0.965	0.780	0.953	0.733	0.969	0.764	0.920	0.749	0.930	0.788	0.917	0.784	0.909	0.727	0.936	0.728
DAE	0.749	0.313	0.701	0.360	0.796	0.319	0.660	0.350	0.687	0.282	0.555	0.318	0.446	0.292	0.707	0.368
VAE	0.765	0.291	0.496	0.327	0.723	0.309	0.566	0.327	0.704	0.260	0.426	0.276	0.311	0.266	0.520	0.345
LAE	0.631	0.305	0.627	0.379	0.739	0.336	0.549	0.365	0.430	0.297	0.446	0.329	0.340	0.294	0.734	0.370
BINet	0.632	0.596	0.643	0.591	0.667	0.615	0.604	0.589	0.633	0.575	0.608	0.599	0.555	0.613	0.601	0.582
GAE	0.363	-	0.511	-	0.374	-	0.284	-	0.480	-	0.296	-	0.308	-	0.560	-
DeepSAD	0.710	-	0.698	-	0.778	-	0.696	-	0.654	-	0.520	-	0.567	-	0.629	-
REPEN	0.682	-	0.411	-	0.744	-	0.387	-	0.546	-	0.380	-	0.287	-	0.485	-
DevNet	0.794	-	0.719	-	0.812	-	0.751	-	0.771	-	0.690	-	0.686	-	0.787	-
PReNet	0.695	-	0.593	-	0.779	-	0.675	-	0.590	-	0.572	-	0.657	-	0.714	-
FEAWAD	0.801	-	0.709	-	0.861	-	0.764	-	0.701	-	0.672	-	0.674	-	0.786	-

technology and adopt eight different business process models [13]. Among these models, two (*Paper* and *P2P*) are handmade, while the remaining six (*Small*, *Medium*, *Large*, *Huge*, *Gigantic*, and *Wide*) are randomly generated using the PLG2 tool [68]. The randomly generated process models have different numbers of activities and varying size ranges. We also use a likelihood graph [34] to generate causally dependent event attributes. Each synthetic log contains 2 to 5 attributes, and we generate a total of $4 * 8 = 32$ synthetic logs that are free of anomalies.

For real-life logs, there are eight commonly used logs, which include the following: i) *Billing*: This log contains events that pertain to the billing of medical services provided by a hospital. ii) *Receipt*: This log contains records of the receiving phase of the building permit application process in an anonymous municipality. iii) *Sepsis*: This log contains events of sepsis cases from a hospital. iv) *Request*: This log contains events related to Requests for Payment. v) *Prepaid*: This log contains events related to prepaid travel costs. vi) *Loan*: This log contains events related to the loan application process of a Dutch financial institute. vii) *Approval*: This log contains information for loan and overdraft approvals processes from submission to eventual resolution (approval, cancellation, or rejection). viii) *Declaration*: This log contains events related to domestic travel declarations.

The summary of statistics for each event log used in the experiments is presented in Table 2. To evaluate the efficacy of our approach, we deliberately introduce ten percent artificial anomalies into both synthetic and real-life logs. This means that ten percent of the traces will be anomalous.

5.5 Experiments on Synthetic Logs

5.5.1 Performance Evaluation

Table 3 reports the performance of weakly supervised business process anomaly detection and interpretation of the cause of anomalies of various methods over eight synthetic logs. The best results are indicated using bold typeface. We have the following observations.

The proposed WAKE framework achieves the best performance on all the synthetic logs in terms of weakly supervised business process anomaly detection and interpretation of the cause of anomalies. In terms of anomaly detection, the F -scores of WAKE are on average 18.63% better than the best weakly supervised anomaly detection method (DevNet), and the F -scores of WAKE are on average 27.48% better than the best deep unsupervised anomaly detection method (DAE). This demonstrates that a few labelled anomalies are well leveraged by WAKE to improve the detection performance. While existing weakly supervised anomaly detection methods are able to exploit a few labelled anomalies, they do not take into account the impact of the temporal nature of the events and the varying length of traces in business processes. WAKE utilises BGRUs to capture the temporal information of events and employs a mask mechanism to deal with varying trace lengths. More importantly, the reconstruction errors of traces, which is widely used in unsupervised anomaly detection, is also incorporated into WAKE to improve detection performance. In terms of interpretation of the cause of anomalies, the F -scores of WAKE are on average 16.16% better than the best deep unsupervised anomaly detection method (BINet).

TABLE 4: F -scores over real-life logs where 'AD' represents weakly supervised business process anomaly detection and 'IA' represents interpretation of the cause of anomalies. The best results are indicated using bold typeface.

Methods	Logs	Billing		Receipt		Sepsis		Request		Prepaid		Loan		Approval		Declaration	
		AD	IA	AD	IA	AD	IA	AD	IA	AD	IA	AD	IA	AD	IA	AD	IA
WAKE		0.940	0.557	0.564	0.780	0.469	0.619	0.934	0.547	0.655	0.768	0.720	0.597	0.845	0.621	0.930	0.722
DAE		0.583	0.492	0.478	0.337	0.307	0.338	0.756	0.409	0.577	0.222	0.313	0.252	0.521	0.446	0.747	0.420
VAE		0.601	0.452	0.363	0.315	0.281	0.328	0.447	0.325	0.260	0.188	0.196	0.212	0.240	0.423	0.603	0.336
LAE		0.701	0.514	0.419	0.295	0.280	0.255	0.715	0.520	0.496	0.261	0.199	0.261	0.483	0.523	0.709	0.570
BINet		0.604	0.536	0.435	0.404	0.342	0.181	0.640	0.511	0.650	0.616	0.494	0.493	0.596	0.583	0.517	0.668
GAE		0.275	-	0.307	-	0.231	-	0.360	-	0.288	-	0.182	-	0.346	-	0.355	-
DeepSAD		0.804	-	0.293	-	0.287	-	0.611	-	0.556	-	0.253	-	0.494	-	0.673	-
REPEN		0.542	-	0.366	-	0.244	-	0.573	-	0.465	-	0.189	-	0.240	-	0.576	-
DevNet		0.743	-	0.528	-	0.297	-	0.850	-	0.595	-	0.269	-	0.619	-	0.863	-
PReNet		0.776	-	0.512	-	0.315	-	0.728	-	0.577	-	0.260	-	0.657	-	0.865	-
FEAWAD		0.780	-	0.457	-	0.328	-	0.815	-	0.563	-	0.300	-	0.553	-	0.847	-

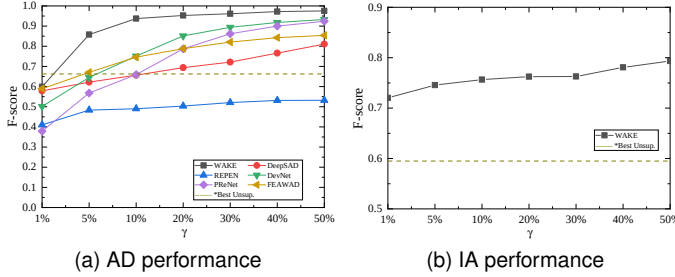


Fig. 4: F -scores under different ratios of labelled anomalies over synthetic logs.

This suggests that the feature encoder which is used to extract feature representations of traces to reflect anomalies can be fine-tuned and well adapted to interpret the cause of the anomalies.

5.5.2 Impact of Degree of Supervision

Next, we evaluate the impact of degree of supervision i.e., different ratios of labelled anomalies γ . F -score is averaged over all synthetic logs.

Fig. 4a shows the F -scores under different ratios of labelled anomalies γ (i.e., γ percent of all anomalous traces are labelled as labelled anomalies) in terms of weakly supervised anomaly detection. The best unsupervised anomaly detection results are shown as dark yellow dashed lines. We can see that all weakly supervised anomaly detection methods behave similarly with their F -scores increasing with γ . As expected, more labelled anomalies can provide more supervised information. None of the weakly supervised methods outperform the unsupervised method with only 1% labelled anomalies. As expected, the unsupervised approaches only consider the essential features of the process, while the existing semi-supervised approaches treat all unlabelled anomalies as normal samples, and the learned detector can be biased by this large number of unlabelled anomalies (99% of anomalies). Our proposed WAKE and FEAWAD outperform the best unsupervised method with only 5% labelled anomalies, confirming the advantage of WAKE in leveraging a few labelled anomalies more efficiently. When γ is between 1% and 10%, the F -score of WAKE increases rapidly, while when $\gamma > 10\%$, the F -score of WAKE barely increases and approaches 1. However, other methods require a higher γ to achieve a relatively good performance (e.g., at least 40% labelled anomalies are required to achieve a F -score of 0.9 for DevNet). This demonstrates that our approach requires fewer labelled anomalies and

saves on human annotation costs. REPEN hardly varies with γ , whose performance is the worst. Even 50% labelled anomalies are not enough to surpass the best unsupervised method, which may be because the distance between the traces does not reflect the nature of the anomaly.

In terms of interpretation of the cause of anomalies, from Fig. 4b, we can see that the F -score of WAKE increases with the γ and is always higher than the best unsupervised method. This illustrates that coarse-grained labels are helpful to obtain more accurate fine-grained labels and the proposed calculation of anomaly scores for each attribute value does indeed mitigate the effect of the difference in the number of possible values of different attributes.

5.6 Experiments on Real-life Logs

5.6.1 Performance Evaluation

To validate the applicability of our method, we conduct experiments on eight real-life logs. The results are shown in Table 4. For the datasets Billing and Declaration, the hidden layer size of WAKE is set to 16.

As can be seen, the proposed WAKE framework is also superior over all the real-life logs in terms of weakly supervised business process anomaly detection and interpretation of the cause of anomalies. In terms of anomaly detection, the F -scores of WAKE are on average 16.17% better than the best weakly supervised anomaly detection method (DevNet), and the F -scores of WAKE are on average 22.21% better than the best deep unsupervised anomaly detection method (DAE). At the same time, for most logs (e.g. *Request* and *Declaration*), there are some weakly supervised anomaly detection methods that perform better than deep unsupervised business process anomaly detection methods. This also indicates that if a limited number of anomalies can be used wisely, it can help a lot in improving the detection results. In terms of interpretation of the cause of anomalies, the F -scores of WAKE are on average 15.49% better than the best deep unsupervised anomaly detection method (BINet), showing that a limited number of labelled anomalies are equally helpful in interpreting the cause of anomalies.

5.6.2 Impact of Degree of Supervision

Next, we evaluate the impact of degree of supervision. F -score is obtained over all real-life logs. The results are shown in Fig. 5a and Fig. 5b in terms of weakly supervised anomaly detection and the interpretation of the cause of anomalies, respectively.

TABLE 5: Ablation study - F -scores for WAKE and its ablated versions in terms of anomaly detection.

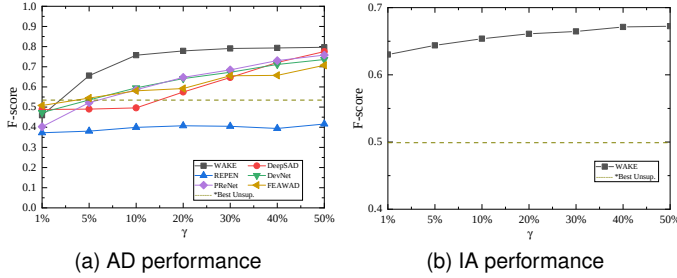
	Paper	P2P	Small	Medium	Large	Huge	Gigantic	Wide	Billing	Receipt	Sepsis	Request	Prepaid	Loan	Approval	Declaration
WAKE	0.965	0.953	0.969	0.920	0.930	0.917	0.909	0.936	0.940	0.564	0.469	0.934	0.655	0.720	0.845	0.930
w/ s	0.653	0.680	0.778	0.461	0.488	0.423	0.426	0.642	0.916	0.285	0.275	0.865	0.417	0.220	0.480	0.900
w/ r	0.919	0.908	0.902	0.886	0.833	0.836	0.823	0.847	0.901	0.545	0.374	0.920	0.558	0.619	0.788	0.914
w/ m	0.507	0.749	0.451	0.646	0.495	0.336	0.535	0.406	0.905	0.536	0.216	0.898	0.483	0.305	0.725	0.866

TABLE 6: Effect of pre-training stage in terms of anomaly detection.

	Paper	P2P	Small	Medium	Large	Huge	Gigantic	Wide	Billing	Receipt	Sepsis	Request	Prepaid	Loan	Approval	Declaration
WAKE	0.965	0.953	0.969	0.920	0.930	0.917	0.909	0.936	0.940	0.564	0.469	0.934	0.655	0.720	0.845	0.930
w/o Pre	0.911	0.904	0.919	0.882	0.868	0.839	0.840	0.871	0.876	0.512	0.270	0.908	0.534	0.632	0.783	0.922

TABLE 7: Effect of pre-training stage in terms of interpretation of the cause of anomalies.

	Paper	P2P	Small	Medium	Large	Huge	Gigantic	Wide	Billing	Receipt	Sepsis	Request	Prepaid	Loan	Approval	Declaration
WAKE	0.780	0.733	0.764	0.749	0.788	0.784	0.727	0.728	0.557	0.780	0.619	0.547	0.768	0.597	0.621	0.722
w/o Pre	0.743	0.696	0.689	0.699	0.742	0.751	0.695	0.708	0.478	0.684	0.591	0.484	0.706	0.526	0.454	0.452

Fig. 5: F -scores under different ratios of labelled anomalies over real-life logs.

Similar to that in Section 5.5.2, our proposed framework WAKE achieves a good anomaly detection performance with only 10% labelled anomalies, which proves that WAKE is also applicable in real life. However, the F -score of each method remains scarcely close to 1 even when 50% of the anomalies are labelled. This is possibly due to some natural anomalies that are detected but not labelled in the datasets. From Fig. 5b, we can see that WAKE is able to better interpret the cause of anomalies, whose F -score exceeds 0.6 even with only 1% labelled anomalies.

5.7 Ablation Study

We investigate the relative importance of each factor of feature representation in terms of anomaly detection for each log. The results are shown in Table 5.

As can be seen, when only one factor is used for anomaly detection, the reconstruction error vector $\mathbf{r}$ performs best, with the exception of the log *Billing*. This demonstrates that the effectiveness of using the reconstruction error, which is a commonly used indicator for unsupervised anomaly detection, as a component of the feature representation of a trace. It also illustrates the ability of our pre-trained autoencoder to reflect the anomalies. However, using only the maximum reconstruction error vector $\mathbf{m}$ results in relatively poor performance, suggesting that relying solely on the maximum reconstruction error for each attribute may not be sufficient to reflect anomalies, and detection results may be biased by special events. Similarly, using only the latent representation $\mathbf{s}$ also results in relatively poor performance, indicating that with few labelled anomalies, the model may easily overfit without the aid of a reconstruction error that is representative of the distance from the normal pattern. The

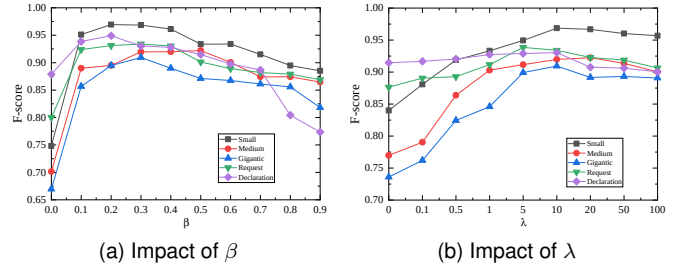


Fig. 6: Parameter analysis in terms of weakly supervised anomaly detection.

proposed WAKE framework (i.e., all three factors were used simultaneously to detect anomalies) has the best performance, suggesting that the combination of the three factors is a more effective strategy, as any significant deviation in any of the three factors can be detected by WAKE.

5.8 Effect of Pre-training Stage

We evaluate the effectiveness of the pre-training stage adopted in our proposed model. We compare WAKE with a model that does not include the pre-training stage (the first stage described in Algorithm 1) in the training procedure.

The results in terms of anomaly detection and interpretation of the cause of anomalies are shown in Table 6 and Table 7, respectively. In terms of anomaly detection, compared with the model without a pre-training stage, WAKE achieves a significant improvement by 5.81% (average) and 7.75% (average) on synthetic logs and real-life logs, respectively. In terms of interpretation of the cause of anomalies, compared with the model without a pre-training stage, WAKE achieves a significant improvement by 4.13% (average) and 10.45% (average) on synthetic logs and real-life logs, respectively. These results show that pre-training is a necessary stage to avoid some bad local minima so that our model can better detect anomalies and interpret their causes.

5.9 Parameter Analysis

5.9.1 Impact of β

Note that the parameter β controls the proportion of labelled anomalies and unlabelled traces in mini-batches in training stage 2, aiming to address sample imbalance. In this section, we investigate the impact of β for weakly

TABLE 8: Computational cost.

	WAKE	DAE	VAE	LAE	BiNet	GAE	DeepSAD	REPEN	DevNet	PReNet	FEAWAD
Running time (s)	1109.94	41.11	200.34	224.19	390.9	1051.74	109.71	45.81	44.81	25.41	105.54

supervised business process anomaly detection by varying the value of β . Fig. 6a illustrates the performance of WAKE on *Small*, *Medium*, *Gigantic* (synthetic logs), *Request* and *Declaration* (real-life logs) under different magnitudes of β in terms of weakly supervised anomaly detection. The labelled anomalies are not oversampled when $\beta = 0$.

As can be seen, the changing pattern of F -scores is basically the same on all logs. Their F -scores increase and then decrease as β increases. When β is small, the model may be biased by a large number of negative samples, i.e. in order to reduce anomaly scores for a large number of unlabelled traces, even some labelled anomalies may be given a very low anomaly score by the trained model. When β is greater than 0.5, the F -scores of WAKE decrease sharply on all logs, indicating that the model is biased by too many labelled anomalies in a mini-batch. At the same time, since the number of labelled anomalies is few, they are oversampled. To traverse all unlabelled traces in a single epoch, the number of batches (i.e., $n_batches_2$) has to be increased, which increases the training time. All in all, a β between 0.2 and 0.5 seems to be a good choice.

5.9.2 Impact of λ

Note that λ is a hyper-parameter to balance the contributions of two parts in the joint loss (i.e., Eq.(16)). When $\lambda = 0$, the anomaly scores generated by the anomaly score generator only have to satisfy a specific distribution (i.e., without minimizing reconstruction errors). In this section, we investigate the impact of λ for weakly supervised anomaly detection by varying the value of λ .

From Fig. 6b, we can see that the changing pattern of F -scores on logs *Small*, *Medium*, *Gigantic* and *Request* is basically the same. Their F -scores increase sharply and then decrease as λ increases which suggests that focusing on the reconstruction errors of traces in the feature encoder can improve the anomaly detection performance of WAKE when the limited number of labelled anomalies is not sufficient to cover all anomaly patterns, but paying too much attention to the reconstruction errors can lead to a poor anomaly detection performance. For the log *Declaration*, λ has less of an effect on it. When additional analyses are performed to examine the characteristics of the log, it can be observed that the limited number of labelled anomalies in the log *Declaration* can cover most anomalous patterns and WAKE performs well even when not concerned with reconstruction errors, but it performs slightly worse when overly concerned with reconstruction errors.

5.10 Computational Cost

The time consumption of each method is presented in Table 8. These results have been averaged across all the datasets.

While our proposed method, WAKE, demonstrates superior performance, it also incurs the highest computational cost. In contrast to other approaches, WAKE necessitates three stages of training. Moreover, the incorporation of GRU further intensifies the computational cost. Similarly,

GAE employs graph neural networks, which contribute to a higher computational cost.

6 CONCLUSIONS AND FUTURE WORK

In this paper, we propose WAKE, a weakly supervised business process anomaly detection framework via a pre-trained autoencoder. Our approach takes advantage of a few labelled anomalies, which are always available in real life, to detect anomalies in unlabelled traces. We propose a novel feature encoder that transforms traces into more meaningful feature representations. The feature encoder is pre-trained to be able to reconstruct normal traces. The feature representations are fed into an anomaly score generator to detect anomalies. To overcome the limitation of data insufficiency for labelled anomalies, the feature encoder and the anomaly score generator are jointly optimized through the training process with a specially designed joint loss. The feature encoder is fine-tuned to interpret the cause of the anomalies. We propose a novel method for interpreting the cause of anomalies by computing anomaly scores that can be compared across attributes, addressing the problem of unfairness caused by differences in the number of possible values for different attributes. Extensive experiments demonstrate that the proposed approach can effectively exploit limited-but-valuable information from a few labelled anomalies. The proposed approach outperforms state-of-the-art approaches with superior performance.

Event logs now contain multimedia data such as videos and images, which capture the scenes upon completion of an event. These diverse sources of information can easily lead to anomalies. In the future, we plan to investigate the fusion of these heterogeneous data types to implement cross-media anomaly detection in business processes.

ACKNOWLEDGMENT

This work is supported by China National Science Foundation (Granted No. 62072301). This work is also partially supported by the Program of Technology Innovation of the Science and Technology Commission of Shanghai Municipality (Granted No. 21DZ1205000).

REFERENCES

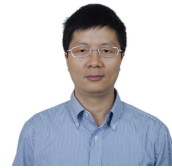
- [1] M. Ebrahim and S. A. H. Golpayegani, "Anomaly detection in business processes logs using social network analysis," *J. Comput. Virol. Hacking Tech.*, vol. 18, no. 2, pp. 127–139, 2022.
- [2] R. Sarno, F. Sinaga, and K. R. Sungkono, "Anomaly detection in business processes using process mining and fuzzy association rule learning," *J. Big Data*, vol. 7, no. 1, p. 5, 2020.
- [3] J. Ko and M. Comuzzi, "Detecting anomalies in business process event logs using statistical leverage," *Inf. Sci.*, vol. 549, pp. 53–67, 2021.
- [4] —, "Keeping our rivers clean: Information-theoretic online anomaly detection for streaming business process events," *Inf. Syst.*, vol. 104, p. 101894, 2022.
- [5] D. Rahmawati, R. Sarno, C. Faticah, and D. Sunaryono, "Fraud detection on event log of bank financial credit business process using hidden markov model algorithm," in *2017 3rd International Conference on Science in Information Technology (ICSITech)*. IEEE, 2017, pp. 35–40.

- [6] C. Linn and D. Werth, "Sequential anomaly detection techniques in business processes," in *Business Information Systems Workshops: BIS 2016 International Workshops, Leipzig, Germany, July 6-8, 2016, Revised Papers 19*. Springer, 2017, pp. 196–208.
- [7] M. G. Armentano and A. A. Amandi, "Detection of sequences with anomalous behavior in a workflow process," in *Database and Expert Systems Applications - 26th International Conference, DEXA 2015, Valencia, Spain, September 1-4, 2015, Proceedings, Part I*, ser. Lecture Notes in Computer Science, Q. Chen, A. Hameurlain, F. Toumani, R. R. Wagner, and H. Decker, Eds., vol. 9261. Springer, 2015, pp. 111–118.
- [8] A. Sureka, "Kernel based sequential data anomaly detection in business process event logs," *CoRR*, vol. abs/1507.01168, 2015.
- [9] P. Hsu, Y. Chuang, Y. Lo, and S. He, "Using contextualized activity-level duration to discover irregular process instances in business operations," *Inf. Sci.*, vol. 391, pp. 80–98, 2017.
- [10] Y. LeCun, Y. Bengio, and G. E. Hinton, "Deep learning," *Nat.*, vol. 521, no. 7553, pp. 436–444, 2015.
- [11] T. Nolle, S. Luetzgen, A. Seeliger, and M. Mühlhäuser, "Analyzing business process anomalies using autoencoders," *Mach. Learn.*, vol. 107, no. 11, pp. 1875–1893, 2018.
- [12] H. T. C. Nguyen, S. Lee, J. Kim, J. Ko, and M. Comuzzi, "Autoencoders for improving quality of process event logs," *Expert Syst. Appl.*, vol. 131, pp. 132–147, 2019.
- [13] T. Nolle, S. Luetzgen, A. Seeliger, and M. Mühlhäuser, "Binet: Multi-perspective business process anomaly classification," *Inf. Syst.*, vol. 103, p. 101458, 2022.
- [14] S. Huo, H. Völzer, P. Reddy, P. Agarwal, V. Isahagian, and V. Muthusamy, "Graph autoencoders for business process anomaly detection," in *Business Process Management - 19th International Conference, BPM, ser. Lecture Notes in Computer Science*, vol. 12875. Springer, 2021, pp. 417–433.
- [15] W. Guan, J. Cao, Y. Gu, and S. Qian, "Grasped: A gru-ae network based multi-perspective business process anomaly detection model," *IEEE Transactions on Services Computing*, pp. 1–13, 2023.
- [16] G. Pang, C. Shen, L. Cao, and A. van den Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 38:1–38:38, 2022.
- [17] L. Ruff, R. A. Vandermeulen, N. Görmitz, A. Binder, E. Müller, K.-R. Müller, and M. Kloft, "Deep semi-supervised anomaly detection," *arXiv preprint arXiv:1906.02694*, 2019.
- [18] G. Pang, L. Cao, L. Chen, and H. Liu, "Learning representations of ultrahigh-dimensional data for random distance-based outlier detection," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018, pp. 2041–2050.
- [19] B. Tian, Q. Su, and J. Yin, "Anomaly detection by leveraging incomplete anomalous knowledge with anomaly-aware bidirectional gans," *arXiv preprint arXiv:2204.13335*, 2022.
- [20] G. Pang, C. Shen, and A. van den Hengel, "Deep anomaly detection with deviation networks," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 353–362.
- [21] G. Pang, C. Shen, H. Jin, and A. v. d. Hengel, "Deep weakly-supervised anomaly detection," *arXiv preprint arXiv:1910.13601*, 2019.
- [22] Y. Zhou, X. Song, Y. Zhang, F. Liu, C. Zhu, and L. Liu, "Feature encoding with autoencoders for weakly supervised anomaly detection," *IEEE Trans. Neural Networks Learn. Syst.*, vol. 33, no. 6, pp. 2454–2465, 2022.
- [23] N. Görmitz, M. Kloft, K. Rieck, and U. Brefeld, "Toward supervised anomaly detection," *J. Artif. Intell. Res.*, vol. 46, pp. 235–262, 2013.
- [24] S. Das, W.-K. Wong, T. Dietterich, A. Fern, and A. Emmott, "Incorporating expert feedback into active anomaly discovery," in *2016 IEEE 16th International Conference on Data Mining (ICDM)*. IEEE, 2016, pp. 853–858.
- [25] T. Huang, P. Chen, and R. Li, "A semi-supervised vae based active anomaly detection framework in multivariate time series for online systems," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 1797–1806.
- [26] Y. Gong, C. Wang, X. Dai, S. Yu, L. Xiang, and J. Wu, "Multi-scale continuity-aware refinement network for weakly supervised video anomaly detection," in *2022 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE, 2022, pp. 1–6.
- [27] J. Wu, W. Zhang, G. Li, W. Wu, X. Tan, Y. Li, E. Ding, and L. Lin, "Weakly-supervised spatio-temporal anomaly detection in surveillance video," *arXiv preprint arXiv:2108.03825*, 2021.
- [28] B. Wan, Y. Fang, X. Xia, and J. Mei, "Weakly supervised video anomaly detection via center-guided discriminative learning," in *IEEE International Conference on Multimedia and Expo, ICME 2020, London, UK, July 6-10, 2020*. IEEE, 2020, pp. 1–6.
- [29] E. A. Elaziz, R. Fathalla, and M. Shaheen, "Deep reinforcement learning for data-efficient weakly supervised business process anomaly detection," *Journal of Big Data*, vol. 10, no. 1, p. 33, 2023.
- [30] F. de Lima Bezerra and J. Wainer, "Algorithms for anomaly detection of traces in logs of process aware information systems," *Inf. Syst.*, vol. 38, no. 1, pp. 33–44, 2013.
- [31] R. Sarno, R. D. Dewandono, T. Ahmad, M. F. Naufal, and F. Sinaga, "Hybrid association rule learning and process mining for fraud detection," *IAENG International Journal of Computer Science*, vol. 42, no. 2, 2015.
- [32] J. Ko and M. Comuzzi, "Online anomaly detection using statistical leverage for streaming business process events," in *Process Mining Workshops - ICPM 2020 International Workshops, Padua, Italy, October 5-8, 2020, Revised Selected Papers*, ser. Lecture Notes in Business Information Processing, S. J. J. Leemans and H. Leopold, Eds., vol. 406. Springer, 2020, pp. 193–205.
- [33] —, "Business process event log anomaly detection based on statistical leverage," in *ITBPM@BPM*, 2021, pp. 7–12.
- [34] K. Böhmer and S. Rinderle-Ma, "Multi-perspective anomaly detection in business process execution events," in *On the Move to Meaningful Internet Systems: OTM 2016 Conferences - Confederated International Conferences: CoopIS, C&TC, and ODBASE 2016, Rhodes, Greece, October 24-28, 2016, Proceedings*, ser. Lecture Notes in Computer Science, C. Debruyne, H. Panetto, R. Meersman, T. S. Dillon, eva Kühn, D. O'Sullivan, and C. A. Ardagna, Eds., vol. 10033, 2016, pp. 80–98.
- [35] A. Rogge-Solti and G. Kasneci, "Temporal anomaly detection in business processes," in *International Conference on Business Process Management*. Springer, 2014, pp. 234–249.
- [36] B. Kang, D. Kim, and S. Kang, "Real-time business process monitoring method for prediction of abnormal termination using knn-based LOF prediction," *Expert Syst. Appl.*, vol. 39, no. 5, pp. 6061–6068, 2012.
- [37] F. Folino, G. Greco, A. Guzzo, and L. Pontieri, "Mining usage scenarios in business processes: Outlier-aware discovery and run-time prediction," *Data Knowl. Eng.*, vol. 70, no. 12, pp. 1005–1029, 2011.
- [38] G. M. Tavares and S. Barbon Jr, "Analysis of language inspired trace representation for anomaly detection," in *International Conference on Theory and Practice of Digital Libraries*. Springer, 2020, pp. 296–308.
- [39] S. B. Junior, P. Ceravolo, E. Damiani, N. J. Omori, and G. M. Tavares, "Anomaly detection on event logs with a scarcity of labels," in *2020 2nd International Conference on Process Mining (ICPM)*. IEEE, 2020, pp. 161–168.
- [40] P. Krajsic and B. Franczyk, "Lambda architecture for anomaly detection in online process mining using autoencoders," in *Advances in Computational Collective Intelligence - 12th International Conference, ICCCI 2020, Da Nang, Vietnam, November 30 - December 3, 2020, Proceedings*, ser. Communications in Computer and Information Science, M. Hernes, K. Wojtkiewicz, and E. Szczerbicki, Eds., vol. 1287. Springer, 2020, pp. 579–589.
- [41] T. Nolle, A. Seeliger, and M. Mühlhäuser, "Unsupervised anomaly detection in noisy business process event logs using denoising autoencoders," in *Discovery Science - 19th International Conference, DS 2016, Bari, Italy, October 19-21, 2016, Proceedings*, ser. Lecture Notes in Computer Science, T. Calders, M. Ceci, and D. Malerba, Eds., vol. 9956, 2016, pp. 442–456.
- [42] P. Krajsic and B. Franczyk, "Variational autoencoder for anomaly detection in event data in online process mining," in *ICEIS (I)*, 2021, pp. 567–574.
- [43] T. Nolle, A. Seeliger, and M. Mühlhäuser, "Binet: Multivariate business process anomaly detection using deep learning," in *Business Process Management - 16th International Conference, BPM 2018, Sydney, NSW, Australia, September 9-14, 2018, Proceedings*, ser. Lecture Notes in Computer Science, M. Weske, M. Montali, I. Weber, and J. vom Brocke, Eds., vol. 11080. Springer, 2018, pp. 271–287.
- [44] W. Guan, J. Cao, Y. Gu, and S. Qian, "Gama: A multi-graph-based anomaly detection framework for business processes via graph neural networks," 2023.
- [45] P. Krajsic and B. Franczyk, "Semi-supervised anomaly detection

- in business process event data using self-attention based classification," *Procedia Computer Science*, vol. 192, pp. 39–48, 2021.
- [46] C. Huang, F. Ye, Y. Zhang, Y. Wang, and Q. Tian, "ESAD: end-to-end deep semi-supervised anomaly detection," *CoRR*, vol. abs/2012.04905, 2020.
- [47] Z. Li, C. Sun, C. Liu, X. Chen, M. Wang, and Y. Liu, "Dual-mgan: An efficient approach for semi-supervised outlier detection with few identified anomalies," *ACM Trans. Knowl. Discov. Data*, vol. 16, no. 6, pp. 107:1–107:30, 2022.
- [48] Z. Feng, J. Tang, Y. Dou, and G. Wu, "Learning discriminative features for semi-supervised anomaly detection," in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP*. IEEE, 2021, pp. 2935–2939.
- [49] W. Sultani, C. Chen, and M. Shah, "Real-world anomaly detection in surveillance videos," in *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 2018, pp. 6479–6488.
- [50] M. Z. Zaheer, A. Mahmood, H. Shin, and S. Lee, "A self-reasoning framework for anomaly detection using video-level labels," *CoRR*, vol. abs/2008.11887, 2020.
- [51] D. Lee, S. Yu, H. Ju, and H. Yu, "Weakly supervised temporal anomaly segmentation with dynamic time warping," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 7355–7364.
- [52] C. Zhang, Q. Wang, T. Liu, X. Lu, J. Hong, B. Han, and C. Gong, "Fraud detection under multi-sourced extremely noisy annotations," in *Proceedings of the 30th ACM international conference on information & knowledge management*, 2021, pp. 2497–2506.
- [53] Y. Zhao, G. Zheng, S. Mukherjee, R. McCann, and A. H. Awadallah, "Admoe: Anomaly detection with mixture-of-experts from noisy labels," *CoRR*, vol. abs/2208.11290, 2022.
- [54] Y. Gao, X. Wang, X. He, Z. Liu, H. Feng, and Y. Zhang, "Alleviating structural distribution shift in graph anomaly detection," in *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*, 2023, pp. 357–365.
- [55] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 16 000–16 009.
- [56] Y. Ye, L. Xia, and C. Huang, "Graph masked autoencoder for sequential recommendation," in *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2023, Taipei, Taiwan, July 23-27, 2023*. ACM, 2023, pp. 321–330.
- [57] Q. Tan, N. Liu, X. Huang, R. Chen, S.-H. Choi, and X. Hu, "Mgae: Masked autoencoders for self-supervised learning on graphs," *arXiv preprint arXiv:2201.02534*, 2022.
- [58] P.-Y. Huang, H. Xu, J. Li, A. Baevski, M. Auli, W. Galuba, F. Metze, and C. Feichtenhofer, "Masked autoencoders that listen," *Advances in Neural Information Processing Systems*, vol. 35, pp. 28 708–28 720, 2022.
- [59] H. Yao, X. Wang, and W. Yu, "Siamese transition masked autoencoders as uniform unsupervised visual anomaly detector," *arXiv preprint arXiv:2211.00349*, 2022.
- [60] M.-I. Georgescu, "Masked autoencoders for unsupervised anomaly detection in medical images," *arXiv preprint arXiv:2307.07534*, 2023.
- [61] N.-C. Ristea, F.-A. Croitoru, R. T. Ionescu, M. Popescu, F. S. Khan, and M. Shah, "Self-distilled masked auto-encoders are efficient video anomaly detectors," *arXiv preprint arXiv:2306.12041*, 2023.
- [62] W. Yu, M. Huang, S. Wu, and Y. Zhang, "Ensembled masked graph autoencoders for link anomaly detection in a road network considering spatiotemporal features," *Information Sciences*, vol. 622, pp. 456–475, 2023.
- [63] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.
- [64] K. Cho, B. van Merriënboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL, A. Moschitti, B. Pang, and W. Daelemans, Eds. ACL, 2014, pp. 1724–1734.*
- [65] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [66] J. R. Quinlan, "Induction of decision trees," *Mach. Learn.*, vol. 1, no. 1, pp. 81–106, 1986.
- [67] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, AIS-TATS 2010, Chia Laguna Resort, Sardinia, Italy, May 13-15, 2010*, ser. JMLR Proceedings, Y. W. Teh and D. M. Titterton, Eds., vol. 9. JMLR.org, 2010, pp. 249–256.
- [68] A. Burattin, "PLG2: multiperspective processes randomization and simulation for online and offline settings," *CoRR*, vol. abs/1506.08415, 2015.



Wei Guan is currently a Ph.D student in the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His research interests include predictive monitoring, anomaly detection and machine learning.



Jian Cao is currently a tenured professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. He is also the director of research institute of network computing and service computing. Dr. Cao's research interests include intelligent data analytics, service computing. He has published more than 300 research papers in prestigious journals and conferences. Currently, he is the distinguished member of CCF and the senior member of IEEE.



Haiyan Zhao is an associate professor affiliated with the University of Shanghai for Science and Technology. She received her Ph.D. degree from Nanjing University of Science and Technology in 2002. She has authored over 80 papers in journals and conferences, and her research interests include data analytics, recommender systems, and business process management.



Yang Gu is currently a Ph.D student in the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His research interests include service computing and data mining.



Shiyou Qian received a PhD degree in computer science from Shanghai Jiao Tong University, China, in 2015. He is currently an associate researcher with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China. His research interests include event matching for content-based publish/subscribe systems, resource scheduling for Hybrid-Cloud, and driving recommendation with vehicular networks.